

Survival Model Evaluation: `chicago_licences`

Fitted with the survival-analysis-pipeline.

Source data	<code>datasets/chicago_licences.csv.gz</code>
Rows	239,721 (82.2% with the ending observed, 17.8% censored)
Start dates	2002-01-02 to 2026-07-31
Evaluation	5 expanding-window temporal folds
Source of figures	<code>reports/chicago_demo/metrics.json</code> , regenerated by the command in section 8

1. Summary

This report evaluates two survival models fitted to `chicago_licences.csv.gz`. It holds 239,721 rows, each observed from its start date. 82.2% have observed endings and 17.8% are censored, still running when observation stopped. Median observed duration, censored rows included, is 904 days, the scale on which every horizon and prediction below sits. The models predict, from what was on file at the start date, how long each row survives.

When evaluating the two models' performance, the apples-to-apples comparison is the fold-mean concordance, the share of row pairs a ranking orders correctly where 0.500 is a coin flip. The pipeline uses walkforward validation. Each of the 5 temporal folds trains both models on one stretch of history and tests them on the next, and the 5 scores average. On concordance the XGBoost accelerated-failure-time (AFT) model scores 0.585 and the Cox proportional hazards baseline 0.592. The Cox baseline scores higher.

To attach an uncertainty range, which says how far the score could reasonably move, the AFT model is also scored with all 143,833 out-of-time test rows pooled into one list. That concordance is 0.576, with a 95% bootstrap interval of 0.574 to 0.578. Section 4 explains the two figures and how to read the interval.

Most of the pooled score reflects a row's `license_description` group, where `license_description` is the grouping column this run was given. Ranking rows by their group's average prediction alone scores 0.614, and comparing only rows inside the same group scores 0.513, close to the coin flip. Section 4 gives the decomposition.

Both models' probabilities lose to a no-skill forecast at 365 days. A no-skill forecast assigns every row the same population-average probability. The limitations section says what remains usable.

Both models are saved in one bundle, which records the Cox baseline as recommended for scoring new rows.

2. Data

Durations are measured in days. These columns hold numeric codes rather than quantities and were forced to categorical: ward, community_area, police_district, zip_code.

Left truncation, where a row was already running when the source's records begin, leaves a recorded start that is not the true start. This pipeline has no delayed-entry handling, so it cannot take a row that entered observation partway through its life, and those rows must be excluded during preparation. Whether they were, and how, is recorded in the dataset's own documentation.

Analyst notes:

For this dataset the preparation step did perform left-truncation exclusion. The portal's history is complete only starting from 2002, so a license already running then shows its first post-2002 renewal as its recorded start, not its true start, and nothing in the data labels these rows. Keeping only licenses whose earliest transaction is a genuine first issue removed 79,690 of them, 23 percent of the licenses downloaded from the portal. What exposed them was a spike of 61,351 supposed starts dated 2002, against roughly 14,000 usual first issues in a normal year. No license in this file starts before 2002. The rule and the counts are recorded in `scripts/run_prepare_chicago.py`.

License types that are short or event-scoped by construction are excluded as well. This includes the Special Event, Pop-Up, and Itinerant variants. That removed 23,042 licenses across 12 types. These permits are intended to be temporary. Their lack of survival is obvious and expected and they do not contain useful general information about which types of business have a greater chance to survive or not. Training on those rows teaches the model to recognize short-term permit types, not to generalize.

License terms are visible in the curves below. The vertical drop at about two years, sharpest in the Peddler License curve, and the smaller steps at term multiples in the others, are terms expiring. A business that does not renew exits at a term boundary, so endings cluster there.

Figure 1 shows Kaplan-Meier survival curves by `license_description`, the coarsest structure in the outcome before any model is fitted.

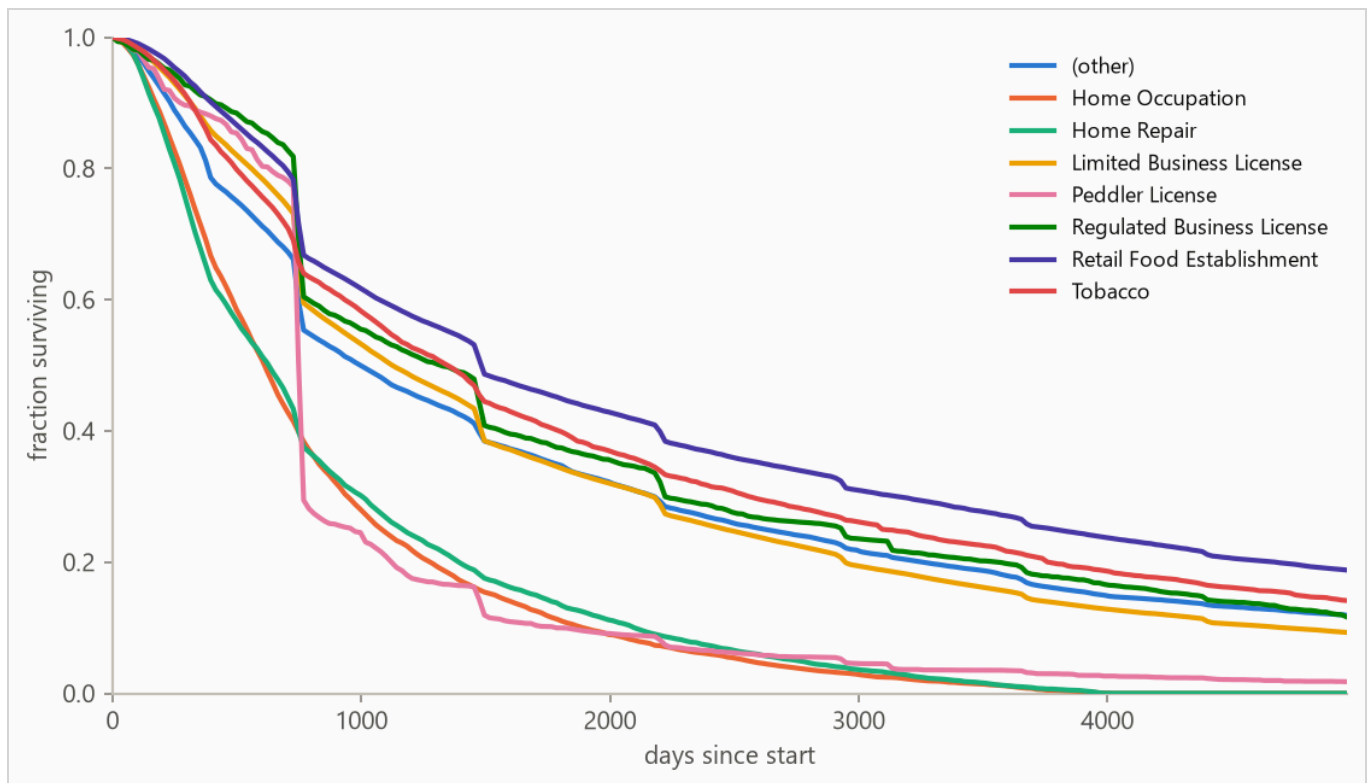


Figure 1. Kaplan-Meier survival curves by `license_description`: the fraction of each group still running at each age, with censored rows counted for as long as they were observed. Groups beyond the seven most frequent, where present, are collapsed into (other) for this plot only.

3. Method

3.1 Model class

The pipeline fits two models on every run. The first is a boosted-tree accelerated failure time (AFT) model, XGBoost with its `survival:aft` objective, built for durations with incomplete observations. An observed ending tells the model the exact lifetime, and a censored row tells it only "at least this long", so every row contributes. It predicts each row's median survival time in days, and a log-normal curve around that median, whose width is fitted once and shared by every row, gives the probability of surviving any horizon. The second is a Cox proportional hazards baseline, the standard linear survival model, fitted with lifelines' `CoxPHFitter` on the same features to answer whether the boosted model was necessary. Its coefficients are fitted under a penalty that pulls them toward no effect, which holds the fit steady when features move together but leaves the uncertainty intervals drawn around those coefficients approximate. The run recommends whichever scores the higher fold-mean concordance.

The two differ in what they assume. The AFT model assumes every row follows the same survival curve run on a faster or slower clock, so features change how fast that clock runs, never the curve's shape. The Cox model

makes no assumption about that shape at all, estimating the curve from the data by counting who was still running at each age. In exchange it assumes each feature multiplies risk by the same factor at every age, which this report does not test. Which set of assumptions suits a dataset cannot be known in advance, which is why both are fitted and scored.

Numeric features pass through, missing values included (the Cox baseline gets train-window median imputation). Text columns are one-hot encoded with the vocabulary refit per training window, so a fold's features reflect only what was on file by its split date.

3.2 Temporal validation and label re-censoring

Evaluation uses 5 expanding-window folds ordered by start date. The earliest 40% of rows is set aside as burn-in and never tested. Each fold trains on every row started before its split date and tests on the next block. A split date can only fall on a date the data contains, so the first fold trains on 95,870 rows, the burn-in rounded to a date boundary (sizes in Table 2).

Training labels are re-censored at each split date. A row started long before a split may have died after it, and its label contains that future, so every post-split death is rewritten as a censoring at the split. Omitting this raises scores by importing the future.

3.3 Selection and calibration

The boosted model's hyperparameters are selected once on the first fold's training window by an inner temporal split, the same past-then-future cut made inside that window. Concordance grades only whether rows are ordered correctly, and a model can order them well while drawing survival curves far too wide or too narrow. So selection is scored on held-out censored log-likelihood instead, which grades the whole predicted distribution against what was observed.

The predictive scale is the width of the boosted model's log-normal curve, one unitless number shared by every row. A larger scale spreads the model's probability over a wider range of lifetimes. Two values of it appear in this run. The first, 1.2, is the setting used while training, where it shapes how the medians are fitted. The second, 0.88, is measured afterwards. With the fitted medians held fixed, it is the width that best matches the held-out outcomes on the training window's most recent stretch, and it is carried to the model refitted on the full window. The training value answers which width trains the best medians. The measured value answers which width matches the outcomes those medians actually got. The two need not agree, and every probability the boosted model reports here uses the measured value.

4. Results

4.1 Discrimination

Table 1. Concordance index by model, pooled over 143,833 test rows with 95% percentile bootstrap intervals over 500 resamples. Higher is better, and 0.500 is a coin flip, and Harrell's C is the standard estimator of it. Fold-mean rows score each of the 5 folds separately and average. The interval resamples test rows with the fitted models held fixed, so it measures scoring precision, not stability across history. The fold spread is the guide to that.

METHOD	CONCORDANCE (HARRELL'S C)	95% INTERVAL
XGBoost AFT (pooled)	0.576	0.574 to 0.578
XGBoost AFT (fold mean)	0.585	not resampled
Cox proportional hazards (fold mean)	0.592	not resampled

Each fold fits its own models. Because the AFT model predicts a survival time in days, and days is a universal unit that exists outside of an individual fold model, its predictions can be pooled into one list while Cox proportional hazards models cannot. A Cox model predicts a hazard score, the risk of ending at any given moment, relative to the average row in the window it was trained on, not universal time units. As a result, Cox scores from different folds cannot be compared in one list. For a fair comparison, the two models must be compared on fold mean concordance, which is each fold scored on its own and the 5 scores averaged.

The pooled row answers a different question. It scores all test rows in one list, which is enough data to attach an uncertainty range as seen in Table 1. The interval is the range the score stayed inside for 95% of resamples of the test rows. A narrow interval means the score is precise on this test set. And an interval that sits wholly above 0.500 means the model beats a coin flip even at its low end.

It is important to determine whether the model does more than recognize which group a row belongs to. In order to assess this, we split the pooled concordance by `license_description`. Re-ranking rows by their group's average replaces every row in a group with the same score, so comparisons only ever run between groups. Evaluating the pooled concordance on those re-ranked rows gives a score of 61.4%. The boosted model scores each row individually instead, which orders rows inside a group. Inside a group this ranking is correct 51.3% of the time, averaged over the 65 groups large enough to score (at least 50 rows and 10 observed endings), each weighted by its number of comparable pairs. When the group-average score matches or beats the model's own pooled score, the model's ranking comes from telling groups apart. When the within-group score sits well above 0.500, the model also orders rows inside a group, which is the part a lookup table of group averages cannot do.

Figure 2 plots the per-fold concordance from Table 2.

Table 2. Per-fold results. Censoring is the share of each fold's test rows whose ending was not observed.

FOLD	SPLIT DATE	TRAIN N	TEST N	CENSORED	AFT	COX
1	2009-05-18	95,870	28,767	7%	0.601	0.595
2	2012-03-27	124,601	28,767	14%	0.559	0.558
3	2013-11-27	153,398	28,767	18%	0.557	0.599
4	2017-08-07	182,164	28,766	29%	0.601	0.592
5	2021-11-02	210,947	28,766	67%	0.610	0.615

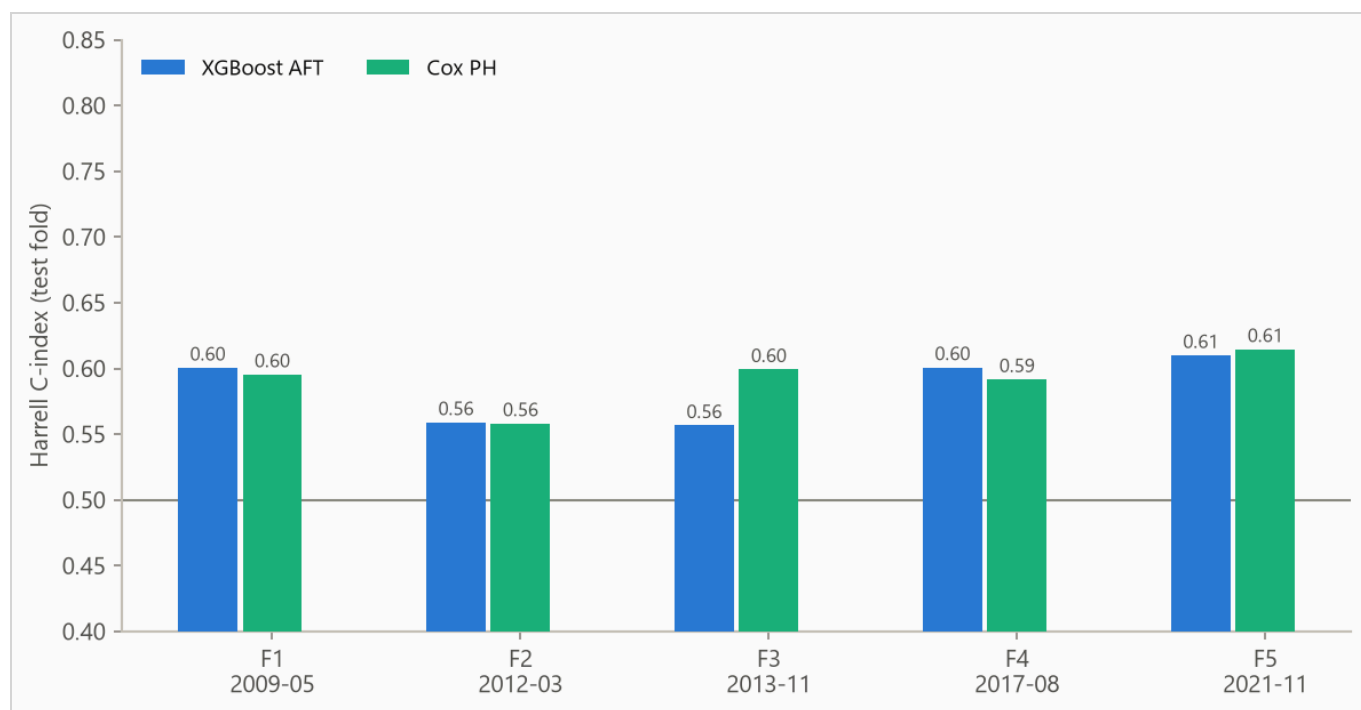


Figure 2. Concordance by fold. The boosted model spans 0.557 to 0.610. Folds differ in censoring mix, so cross-fold comparisons are indicative rather than exact.

4.2 Calibration

A majority of this report focuses on concordance. But concordance grades only whether models produce the correct ordering when comparing two rows. It cannot be used to assess the absolute accuracy of the predicted probabilities. To assess the absolute accuracy, we use the Brier score, the mean squared error of a probability forecast (lower is better). Censored rows can bias the Brier score, so we use the inverse probability of censoring weighting (IPCW) to compensate. At each horizon in Table 3, the models predict each row's chance of still running that length of time. The no-skill forecast predicts the same chance for every row, the share of the whole population still running at that horizon, estimated with the Kaplan-Meier method so censored rows count for as long as they were observed.

Table 3. IPCW Brier score by horizon. Lower is better, and the no-skill column is the score to beat, so a model above it adds nothing at that horizon.

HORIZON	AFT	COX	NO-SKILL FORECAST
365 days	0.095	0.088	0.081
1095 days	0.238	0.231	0.250
1825 days	0.217	0.212	0.227

Figure 3 plots predicted against observed survival at 1095 days for both models. Points falling on both sides of the diagonal are noise. Points falling consistently on one side are bias in that direction.

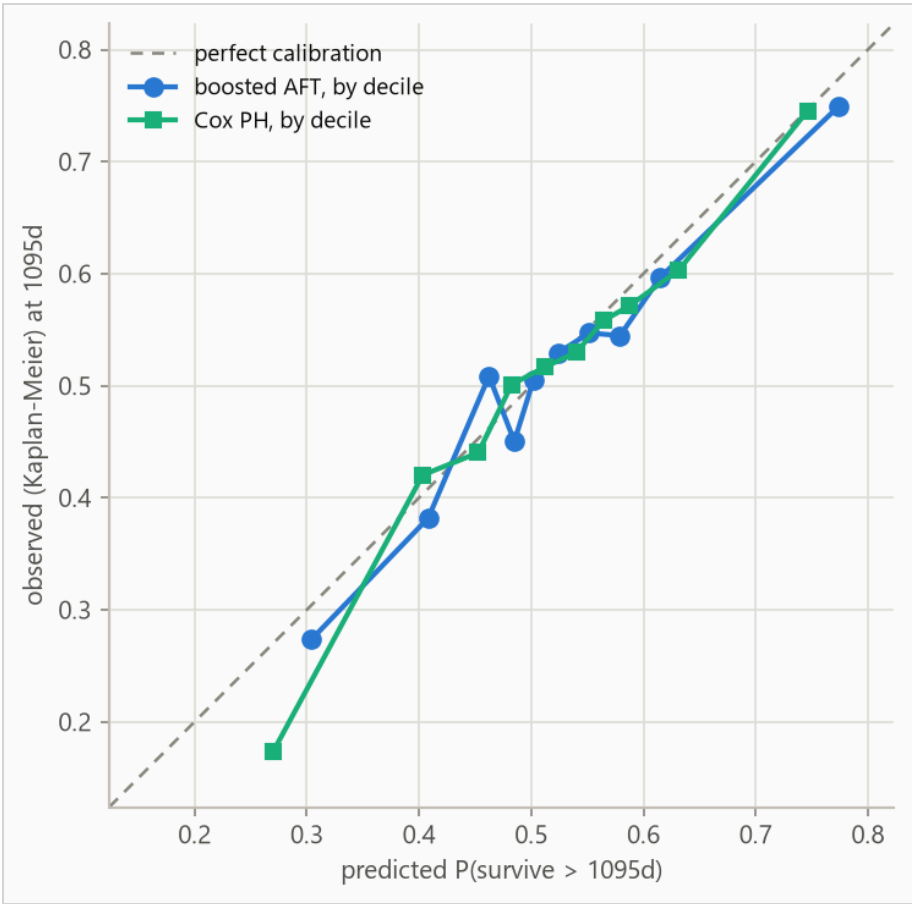


Figure 3. Predicted against observed survival at 1095 days for both models, each binned on its own predicted deciles. Observed frequencies are Kaplan-Meier estimates within each bin, so censored rows contribute correctly. The largest deviation is 0.046 in the boosted model's decile 3 and 0.096 in the Cox baseline's decile 1. Deviations are probabilities. Deciles are cut on predicted value and hold 12,787 to 16,030 rows each. In small or heavily censored deciles the observed value carries more uncertainty than three decimals suggest.

5. Feature analysis

5.1 The boosted model

The scores above grade how well each model ranks rows and how accurate its probabilities are, not which inputs it used. A feature attribution answers that. For one row and one feature, it is that feature's contribution to the difference between that row's prediction and the model's average prediction. The method used here is SHAP (SHapley Additive exPlanations).

Attributions sit on the log scale of survival time, so an attribution is a multiplier on predicted time rather than a number of days. An attribution of +0.3 multiplies that row's predicted survival time by about 1.35, and a negative one shortens it. That conversion follows from the scale and says nothing about this dataset.

Averaging a feature's attributions by size, ignoring sign, says how much that feature moves predictions across the explanation sample, a random subset of the final model's training rows drawn for the attribution computation. A feature high in that average moves predictions a lot, and one low in it barely moves them.

Attributions are descriptive and in-sample, computed on the final boosted model's own training rows. Correlated features split credit by the model's internal choices as much as by the data, so directions are more trustworthy than magnitudes.

Figure 4 ranks the strongest features by that average, and Figure 5 shows, for each of those features, which direction it pushed and how much that varied from row to row.

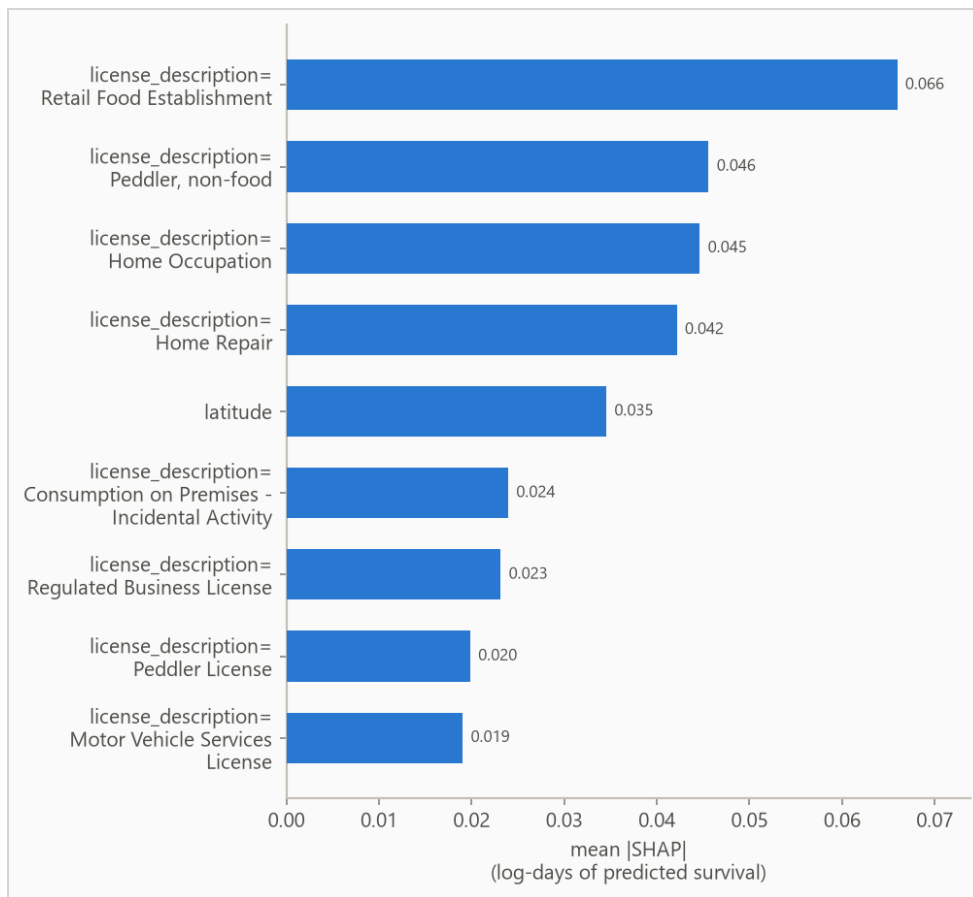


Figure 4. The strongest features, ranked by the average size of their attributions across the explanation sample. Size is taken without regard to sign, so this says how much each feature moves predictions and not which way it moves them; the direction is in Figure 5. The scale is the same log scale of survival time, so a feature averaging 0.10 moves predicted survival time by about a tenth on a typical row. A yes/no flag from a text column keeps its column name in the label. Features below the strongest few are not drawn.

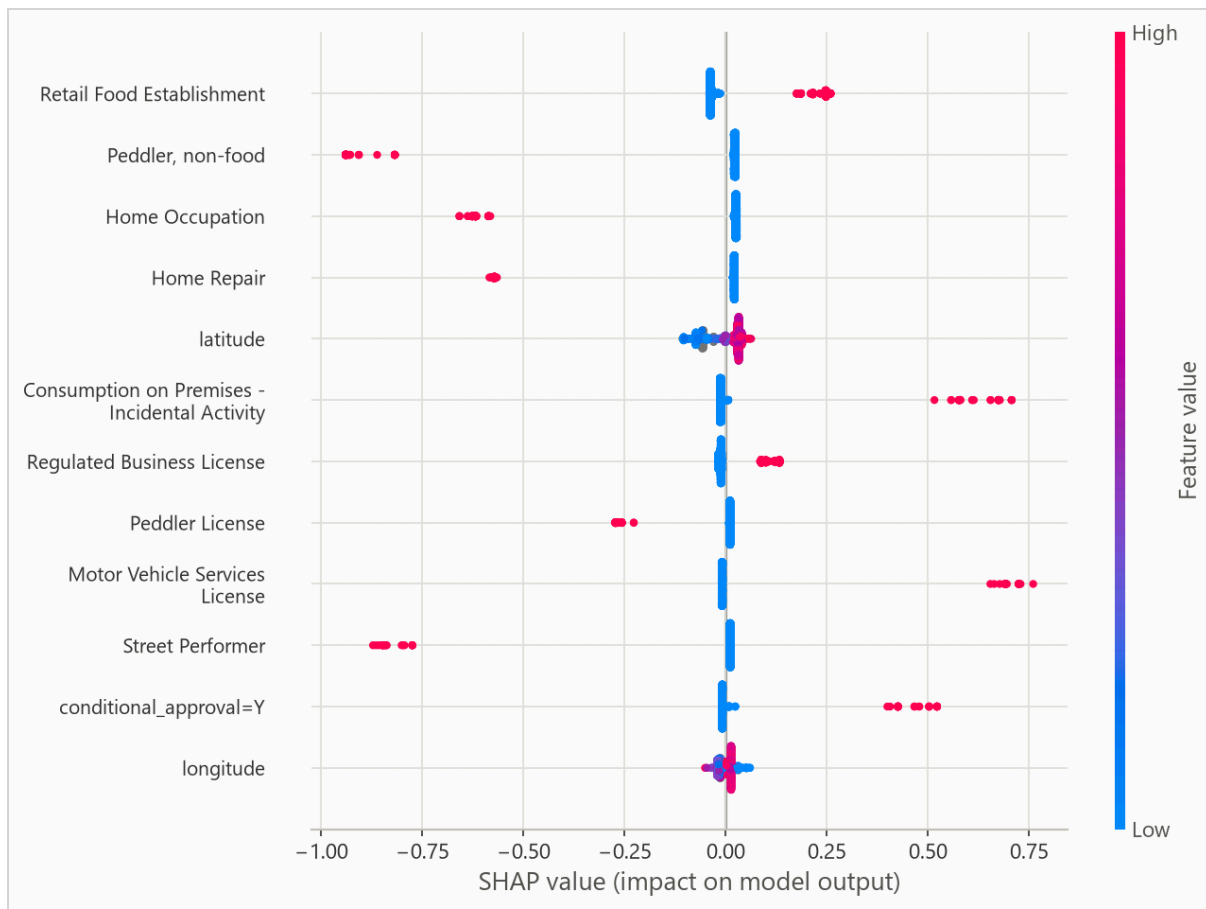


Figure 5. One row per feature, one dot per row of the explanation sample. The dot's position is the attribution that feature earned for that row, so dots left of zero shortened its predicted survival time and dots right of it lengthened. The dot's color is that row's value of the feature, red high and blue low, so a row whose reds sit left of its blues is a feature whose high values shorten survival. How far the dots spread says how much the feature's effect varies from row to row, and a feature pinned at zero did nothing. Single attributions run wider than the averages in Figure 4, since averaging over rows shrinks them. A yes/no flag from a text column keeps its column name in the label and reads the same way, with red meaning the row has that value.

5.2 The Cox baseline

The Cox baseline's drivers are its coefficients, reported as hazard ratios. A hazard ratio is the factor by which one unit of a feature multiplies the hazard and a hazard is the risk of an ending at a given age. The factor is the same at every age, so a ratio above 1 shortens survival. Figure 6 plots the strongest covariates.

Figure 4 describes the boosted model and Figure 6 the Cox baseline, two different models read by two different methods, so they need not name the same features, and neither is a verdict on the data.

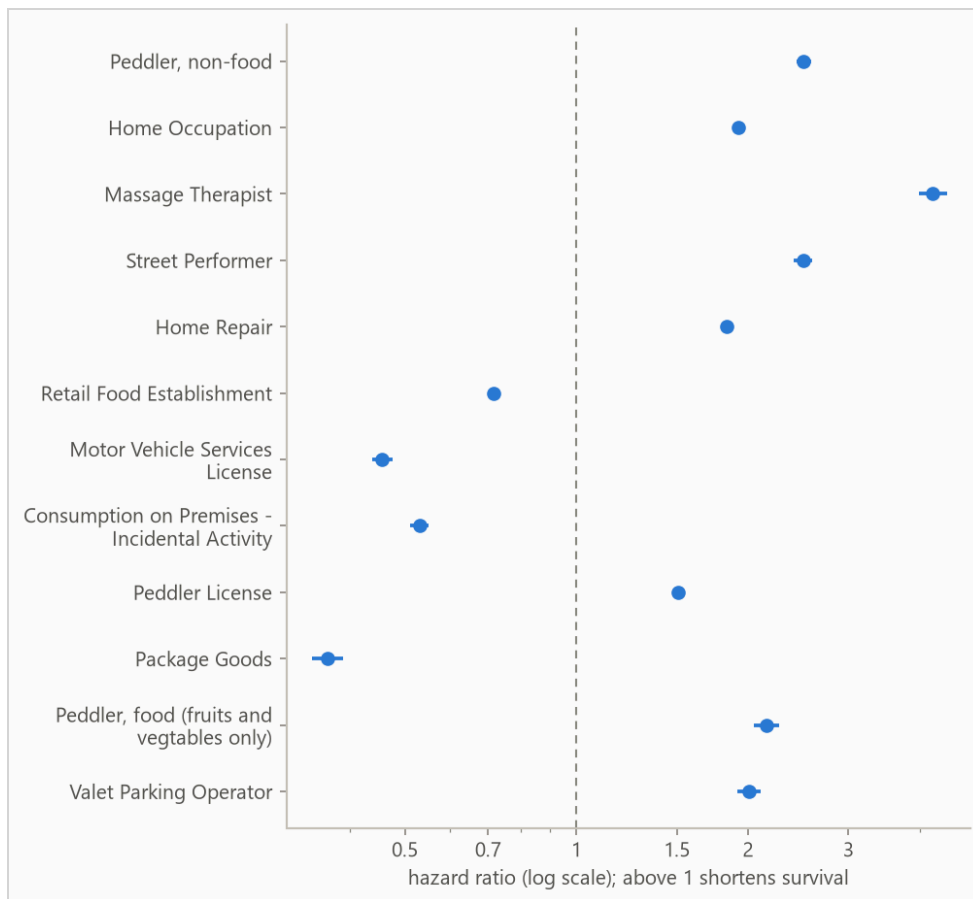


Figure 6. The Cox Proportional Hazards model's strongest covariates with their 95% intervals on a log axis. Dots right of the line shorten survival and left lengthen it. The covariate whose effect the data pins down most firmly sits at the top, and only the strongest few are drawn. Ratios for `license_description` are relative to Accessory Garage, the reference level, chosen alphabetically and given no bar of its own.

6. Interpretation

Analyst notes:

The comparison worth acting on is the model choice. The Cox baseline edges the boosted model on the fold mean, 0.592 to 0.585, and the honest reading is that on this problem a penalized linear model is enough. Day-one paperwork mostly identifies risky license categories. Ranking rows by their category's mean prediction alone scores 0.614 against the model's pooled 0.576, and comparisons within a category score 0.513, barely above chance. So, for example, a Home Repair business correlates with short life, but the metadata says almost nothing about which handyman outlasts the others. Blurring the model's predictions to category averages ranks businesses better than the predictions themselves, so a plain table of category averages is nearly enough on this data.

The one-year probabilities should not drive decisions. Both models lose to the no-skill forecast on Brier score at that horizon, the boosted model at 0.095 and the recommended Cox baseline at 0.088 against the reference's 0.081. Treat either model as an ordering tool at short horizons rather than a probability source there.

Fold 3, the weakest window in the per-fold table, drops for a measurable reason. Its test block sits across a shift in the city's category mix, and the shift is administrative rather than economic. The city stopped issuing the Home Occupation and Home Repair license types in 2012, the same year Regulated Business License first appears with a one-year spike of issues. The change is the city relabeling businesses, not the businesses themselves changing.

The two retired categories carry 11 percent of this fold's training rows and stop appearing in the test block entirely. Regulated Business License nearly triples its share. Another 2 percent of test rows fall in categories the training window never saw. Re-selecting hyperparameters on that fold's own window closes about a third of its gap to the Cox baseline on that fold. This demonstrates that a fold that drops as this one did is worth investigating against the composition of its test block before being dismissed as model instability. The measurements in this and the previous paragraph are recomputed by

```
scripts/run_fold3_investigation.py .
```

The printed bootstrap interval is narrower than the real uncertainty. Licenses in the same category and the same stretch of time tend to fail together, and the interval resamples rows as if they were independent. The per-fold spread in the results section is the better guide to how the score moves across history.

7. Limitations

Both models' absolute probabilities are not usable at 365 days. Both models lose to the no-skill forecast on the censoring-weighted Brier score there, and Table 3 grades each model separately. Ranking and probability quality are separate, so ordering rows is still supported even at the horizons where the probabilities lose.

Censoring may be informative. The evaluation assumes rows stop being observed for reasons unrelated to their risk. If the rows that drop out of observation are the ones about to end, the survival estimates come out too high.

Analyst notes:

Endings near the cutoff are provisional. Where no cancellation exists, a license's end can only be its latest recorded expiry. Inevitably, a license whose term ran out shortly before the cutoff but renewed late cannot be told apart from a real closure. As such, it is recorded as an observed closure. Deaths dated close to

the cutoff therefore include some licenses that will turn out to have survived. The only way to know is to continually repull and update the dataset as time passes.

The boosted model gives every row one curve shape. The predictive scale is a single fitted number, so the model runs a row's clock faster or slower but never changes the curve's shape. Per-row width is future work.

Harrell's concordance is biased under heavy censoring, usually upward. It scores only the pairs where the earlier ending is known, and under heavy censoring those pairs over-represent short observed lifetimes. The most censored test window is 67% censored, so read its rows in Table 2 with the most doubt.

8. Reproducing this run

Python 3.11 or later. From the repository root:

```
python scripts/run_fit_evaluate.py --data datasets/chicago_licences.csv.gz --name chicago_licences
python scripts/run_build_report.py --run reports/chicago_demo
```

Every number is read from the run's `metrics.json` at build time, and a figure the prose never cites fails the build. `requirements.txt` pins the exact versions the committed numbers came from.

Generated from `reports/chicago_demo/metrics.json` by `scripts/run_build_report.py --run`. 239,721 rows, 143,833 out-of-time test rows.